



JS



```
1 // JavaScript
3 const greeting = 'Hello, JavaScript!';
4
5 function greet(name) {
6   return `Hello, ${name}!`;
7 }
8
9 const message = greet('Developer');
10 console.log(message);
11
```



{ }

</>



OUR LADY OF FATIMA UNIVERSITY - ANTIPOLO
COLLEGE OF COMPUTER STUDIES



LESSON 3-5

JAVASCRIPT CONDITIONS, LOOPS AND STRINGS



OUR LADY OF FATIMA UNIVERSITY - ANTIPOLO
COLLEGE OF COMPUTER STUDIES



LESSON 3

JAVASCRIPT CONDITIONS





JavaScript Conditions



JavaScript Conditions are statements that allow a program to make decisions based on whether a condition is true or false.

Think of it like this:

- If it rains → bring an umbrella
- Else → go outside normally

In JavaScript, conditions help control what code will run depending on certain situations.

1. if statement
2. if...else statement
3. if...else if...else
4. switch statement

In JavaScript, These are also part of JavaScript conditional logic and decision-making:

1. Ternary Operator (? :)
2. Switch Statement
3. Booleans
4. Logical Operators



JavaScript if Statement



Executes a block of code if the condition is true.

```
let age = 18;

if (age >= 18) {
  console.log("You are allowed to vote.");
}
```

Common Use:

Used when you need to check one condition and execute code only if it is true.

Example Uses:

- Checking if a user entered the correct password
- Checking if a student passed an exam
- Checking if stock is available





JavaScript if...else statement



Executes one block if true, another block if false.

```
let score = 70;

if (score >= 75) {
  console.log("Passed");
} else {
  console.log("Failed");
}
```

Common Use:

Used when there are two possible outcomes.

Example Uses:

- Login success or failure
- Pass or fail result
- Online or offline status



JavaScript if...else if...else statement



Used when there are multiple conditions to check.

```
let grade = 85;

if (grade >= 90) {
  console.log("Excellent");
} else if (grade >= 80) {
  console.log("Very Good");
} else if (grade >= 75) {
  console.log("Good");
} else {
  console.log("Failed");
}
```

Common Use:

Used when checking multiple ranges or conditions.

Example Uses:

- Grading systems
- Product discounts
- User roles



JavaScript Switch statement



Used when checking multiple possible values of one variable.

```
let day = "Monday";
```

```
switch(day) {  
  case "Monday":  
    console.log("Start of the week");  
    break;  
  case "Friday":  
    console.log("Weekend is near");  
    break;  
  default:  
    console.log("Regular day");  
}
```

Common Use:

Used when comparing one variable against many possible exact values.

Example Uses:

- Selecting menu options
- Days of the week
- User roles
- Game difficulty levels

Why break is important:

Without break, JavaScript continues executing the next cases.



Why Conditions are Important



Conditions make websites and applications interactive by allowing them to:

- Validate login credentials
- Check form inputs
- Show error messages
- Display different content to users
- Create game logic
- Automate decisions

JavaScript conditions help your program decide what action to take based on different situations.





JavaScript Ternary Operator



A shorter way of writing an if...else statement.

```
condition ? expressionIfTrue : expressionIfFalse;
```

```
let age = 20;
```

```
let message = (age >= 18) ? "Adult" : "Minor";
```

```
console.log(message);
```

Common Use:

Used for short and simple decisions.

Example Uses:

- Displaying login status
- Assigning labels
- Quick validation



JavaScript Booleans Operator



A Boolean can only have two values:

- True
- False

Used to check conditions.

```
let isStudent = true;
```

```
if (isStudent) {  
  console.log("Access granted");  
}
```

Common Use:

Used to track states or conditions.

Example Uses:

- Is user logged in?
- Is item available?
- Is form submitted?



OUR LADY OF FATIMA UNIVERSITY - ANTIPOLO
COLLEGE OF COMPUTER STUDIES



LESSON 4

JAVASCRIPT LOOPS





JavaScript Loops



Loops are used to repeat a block of code multiple times until a condition is met. They help automate repetitive tasks in websites and web applications.

Why loops are important in websites:

- Saves time by reducing repetitive code
- Dynamically displays content
- Handles large amounts of data
- Useful for forms, e-commerce, dashboards, and games





JavaScript For Loops



1. For loop

Used when you know how many times you want to repeat something.

Syntax:

```
for (initialization; condition; increment/decrement) {  
    // code to execute  
}
```

Example:

```
for (let i = 1; i <= 5; i++) {  
    console.log("Number: " + i);  
}
```

Common Uses:

- Display numbers in sequence
- Repeating actions a fixed number of times
- Creating tables dynamically
- Looping through arrays with known length



JavaScript While Loops



2. While loop

Used when the number of repetitions is unknown and depends on a condition.

Syntax:

```
while (condition) {  
  // code  
}
```

Example:

```
let i = 1;
```

```
while(i <= 5){  
  console.log(i);  
  i++;  
}
```

Common Uses:

- User login attempts
- Loading data until condition is met
- Game loops
- Repeating tasks until user stops



JavaScript Do-While Loops



3. Do-While loop

Similar to while, but it runs the code at least once before checking the condition.

Syntax:

```
do {  
    // code  
} while(condition);
```

Example:

```
let i = 1;  
  
do {  
    console.log(i);  
    i++;  
} while(i <= 5);
```

Common Uses:

- Menu-driven programs
- Input validation
- Asking users for required information



JavaScript For...of Loops



4. for...of Loop

Used to loop through values of arrays, strings, or iterable objects.

Syntax:

```
for (let item of iterable) {  
  // code  
}
```

Example:

```
let fruits = ["Apple", "Mango", "Orange"];
```

```
for(let fruit of fruits){  
  console.log(fruit);  
}
```

Common Uses:

- Displaying array items
- Reading string characters
- Processing lists of data



JavaScript For...in Loops



5. for...in Loop

Used to loop through object properties.

Syntax:

```
for (let key in object) {  
  // code  
}
```

Example:

```
let student = {  
  name: "John",  
  course: "IT",  
  year: 2  
};
```

```
for(let key in student){  
  console.log(key + ": " + student[key]);  
}
```

Common Uses:

- Reading object data
- Form data handling
- Displaying user information



JavaScript Break Statement



Break Statement

Stops the loop immediately.

Syntax:

```
for(let i = 1; i <= 10; i++){  
  if(i == 5){  
    break;  
  }  
  console.log(i);  
}
```

Common Uses:

- Stop searching when item is found



JavaScript Continue Statement



Continue Statement

Skips the current iteration and moves to the next loop cycle.

Syntax:

```
for(let i = 1; i <= 5; i++){  
  if(i == 3){  
    continue;  
  }  
  console.log(i);  
}
```

Common Uses:

- Skip invalid values during processing



JavaScript Control Flow



Control Flow refers to the order in which JavaScript executes statements in a program. It determines what code runs, when it runs, and under what conditions.

By default, JavaScript executes code from top to bottom, but control flow statements allow programs to make decisions, repeat tasks, or skip certain parts of code.

Types of JavaScript Control Flow

1. Sequential Flow
2. Conditional Statements (Decision Making)
3. Loops (Repetition)
4. Jump Statements





JavaScript Sequential Flow



1. Sequential Flow

Code runs line by line in order.

```
let name = "Arjay";  
console.log("Hello " + name);  
console.log("Welcome to JavaScript!");
```

Common Use:

- Displaying messages
- Running basic instructions step by step





JavaScript Condition Statement



2. Condition Statement (Decision Making)

Used to execute code based on conditions.

1. if Statement
2. if...else
3. else if
4. Switch

Common Use:

- Login validation
- Grading systems
- Menu selections
- User role checking



JavaScript Loops (Repetition)



3. Loops (Repetition)

Used to repeat code multiple times.

```
for(let i = 1; i <= 5; i++){  
  console.log(i);  
}
```

Other loops:

- While
- do...while
- for...of
- for...in

Common Uses:

- Displaying lists
- Processing arrays
- Repeating tasks





JavaScript Jump Statement



Used to alter normal flow.

1. Break

Stops a loop or switch statement.

```
for(let i=1; i<=10; i++){  
  if(i === 5){  
    break;  
  }  
  console.log(i);  
}
```

2. continue

Skips current iteration.

```
for(let i=1; i<=5; i++){  
  if(i === 3){  
    continue;  
  }  
  console.log(i);  
}
```

3. return

Stops function execution and returns a value.

```
function add(a,b){  
  return a + b;  
}  
console.log(add(5,3));
```



Why Control Flow is Important



Control flow makes websites and applications interactive by allowing them to:

- Make decisions
- Repeat tasks
- Handle user inputs
- Manage application logic
- Improve automation

Example in Websites:

- Checking if a user entered the correct password
- Displaying products dynamically
- Processing online forms
- Creating game logic
- Managing navigation menus





OUR LADY OF FATIMA UNIVERSITY - ANTIPOLO
COLLEGE OF COMPUTER STUDIES



LESSON 5

JAVASCRIPT STRINGS



JavaScript Strings



JavaScript Strings are used to store and work with text (letters, words, symbols, sentences).

How to Create Strings

A string is written inside quotes:

```
let name = "Arjay";  
let course = 'Information Technology';
```

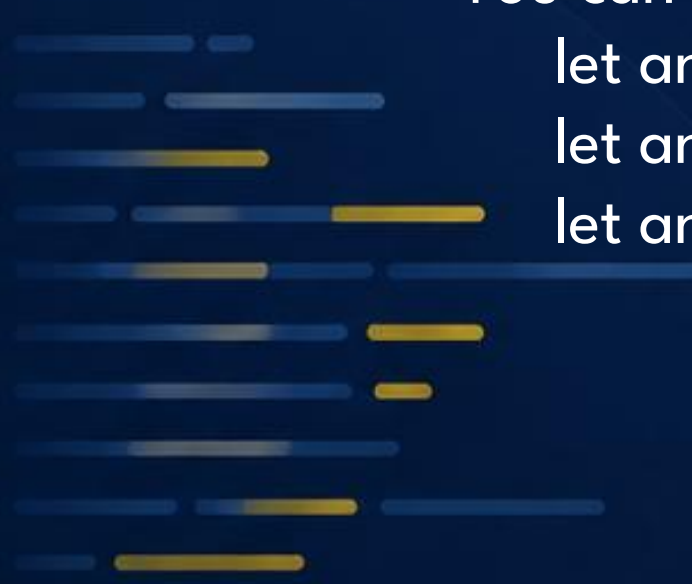
You can use:

Double quotes " " -

Single quotes ' ' -

You can use quotes inside a string, as long as they don't match the quotes surrounding the string:

```
let answer1 = "It's alright";  
let answer2 = "He is called 'Johnny'";  
let answer3 = 'He is called "Johnny"';
```





JavaScript Strings Operations



1. Concatenation (Combining Strings)

Use + to combine text.

```
let firstName = "Arjay";  
let lastName = "Cristobal";
```

```
let fullName = firstName + " " + lastName;  
console.log(fullName);
```

Output:
Arjay Cristobal

2. String Length

Counts the number of characters.

```
let text = "JavaScript";  
console.log(text.length);
```

Output:
10



JavaScript Strings Operations



3. Changing Case

Convert text to uppercase or lowercase.

```
let word = "Hello";
```

```
console.log(word.toUpperCase()); // HELLO  
console.log(word.toLowerCase()); // hello
```

4. Finding Text

Use includes() to check if a word exists.

```
let sentence = "I love coding";
```

```
console.log(sentence.includes("coding"));
```

Output:
true



JavaScript Strings Operations



5. Extracting Part of a String

Use slice().

```
let language = "JavaScript";  
  
console.log(language.slice(0,4));
```

Output:
Java

6. Replacing Text

Use replace().

```
let text = "I like cats";  
  
console.log(text.replace("cats", "dogs"));
```

Output:
I like dogs

Template Literals (Modern Way)

Use backticks `` to insert variables easily.

```
let name = "Arjay";  
let age = 25;  
  
console.log(`My name is ${name} and I am ${age} years old.`);
```





JavaScript References



- https://www.w3schools.com/js/js_operators.asp
- https://www.w3schools.com/js/js_if.asp
- https://www.w3schools.com/js/js_loops.asp
- https://www.w3schools.com/js/js_strings.asp

